

Jegyzőkönyv

Féléves beadandó

Macskamenhely

Név: **Vay Dominika**

Neptunkód: **VM3DIR**

Időpont: **2025.11.02.**

Miskolc, 2025

Tartalom

BEVEZETÉS	3
A PROJEKT - MACSKAMENHELY	3
ROOM ORM	4
Adatmodell.....	4
A Data Access Object.....	4
Adatbázis.....	5
ViewModel.....	7
Függőségkezelés (Dependency Injection)	7
Adatok, UI megjelenítés	7
Coroutines és Flow	8
Architektúra	8
Design	8
Színpaletta.....	8
Téma konfigurációja	9
Betűtípusok	10
UI komponensek	10
Képernyőképek	12
Főoldal	12
Macska hozzáadás/szerkesztés form.....	13
Figyelmeztető üzenet	14
Adoptált macskák oldal.....	15

BEVEZETÉS

A féléves projektem egy macskamenhely üzemeltetéséről szól, ahol hozzá tudunk adni cicákat, szerkeszteni tudjuk őket, törölni, illetve örökbefogadni. Örökbefogadott cicákat vissza is tehetünk “várólistára”. Az alkalmazásom elkészítéséhez az **Android Studio** alkalmazást használtam. **Kotlin** nyelvben írtam meg, ami egy erősen típusos, objektumorientált programozási nyelv, ami könnyedén lehetővé tette számomra, hogy egy jól struktúrált, könnyen átlátható és kezelhető programot hozhassak létre. A project képes adatok tárolására és kezelésére is a **Room ORM** segítségével, amely egy leegyszerűsített SQLite adatbázis használatát teszi lehetővé Android alkalmazásokban. A Room az SQLite fölött működik és elrejt az SQL kódolás nagy részét, ezt az adatbázist Kotlin vagy Java osztályokon keresztül érhetjük el.

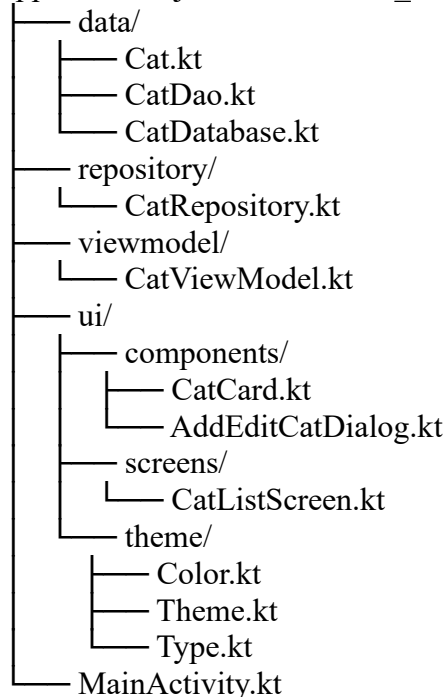
A PROJEKT – MACSKAMENHELY

Github repository: https://github.com/vaydominika/VM3DIR_MobilProg

Kiegészíteném annyival, hogy az AI nagyon aranyos, és jól összeszedett Readme fájlokat tud készíteni miután átolvasta a projektet, be is tettem, a githubon látszódik!

FÁJLSTRUKTÚRA

app/src/main/java/com/vm3dir_mobilprog_catshelter/



ROOM ORM

A projektben három fő komponens található ehhez, az adatmodell (entity), az adatelérési réteg (DAO - Data Access Object) és az adatbázis konfiguráció (maga a Database).

A fájlok nevei és kiterjesztései:

- Az adatmodellhez: Cat.kt.
- Az adateléshez: CatDao.kt.
- A konfigurációhoz: CatDatabase.kt.

Adatmodell

```
@Entity(tableName = "cats")
data class Cat(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,
    val name: String,
    val age: Int,
    val breed: String,
    val color: String,
    val gender: String,
    val description: String,
    val arrivalDate: String,
    val medicalNotes: String,
    val isAdopted: Boolean = false,
    val adoptionDate: String? = null,
    val specialNeeds: Boolean = false
) : Serializable
```

Az `@Entity(tableName = "cats")` rész jelzi a Room-nak, hogy ez az osztály reprezentál egy adatbázist, a `tableName` megadja a tábla nevét.

A `@PrimaryKey(autoGenerate = true)` rész megadja az elsődleges kulcsot, ami itt az `id` lesz, ami automatikusan generálódik majd, ezt az `autoGenerate` rész adja meg.

A `Serializable` pedig megadja, hogy a `Cat` objektumok sorozatba írhatók legyenek, tehát átadhatók más komponensek között.

A Data Access Object

```
@Dao
interface CatDao {
    @Query("SELECT * FROM cats WHERE isAdopted = 0 ORDER BY name ASC")
    fun getAllAvailableCats(): Flow<List<Cat>>
```

```
@Query("SELECT * FROM cats WHERE isAdopted = 1 ORDER BY adoptionDate DESC")
fun getAllAdoptedCats(): Flow<List<Cat>>
```

```
@Query("SELECT * FROM cats ORDER BY name ASC")
fun getAllCats(): Flow<List<Cat>>
```

```
@Query("SELECT * FROM cats WHERE id = :catId")
suspend fun getCatById(catId: Long): Cat?
```

```
@Insert
suspend fun insertCat(cat: Cat): Long
```

```
@Update
suspend fun updateCat(cat: Cat)
```

```
@Delete
suspend fun deleteCat(cat: Cat)
```

```
@Query("UPDATE cats SET isAdopted = 1, adoptionDate = :adoptionDate WHERE id = :catId")
suspend fun markAsAdopted(catId: Long, adoptionDate: String)
```

```
@Query("UPDATE cats SET isAdopted = 0, adoptionDate = NULL WHERE id = :catId")
suspend fun markAsAvailable(catId: Long)
}
```

A `@Dao` jelzi a Room-nak, hogy ez egy DAO, amiben adatbázis műveletek szerepelnek.

A `@Query` kell az SQL lekérdezések végrehajtásához.

Megvalósítja a CRUD műveleteket a(z) `@Insert`, `@Update`, `@Delete` segítségével.

A `suspend` kulcsszó egy Kotlin függvény, aszinkron végrehajtáshoz szükséges, így nem foglaljuk le a fő szálát.

Adatbázis

```
@Database(
    entities = [Cat::class],
    version = 3,
    exportSchema = false
)
abstract class CatDatabase : RoomDatabase() {
    abstract fun catDao(): CatDao

    companion object {
        @Volatile
        private var INSTANCE: CatDatabase? = null
    }
}
```

```

fun getDatabase(context: Context): CatDatabase {
    return INSTANCE ?: synchronized(this) {
        val instance = Room.databaseBuilder(
            context.applicationContext,
            CatDatabase::class.java,
            "cat_database"
        )
        .addMigrations(MIGRATION_1_2, MIGRATION_2_3)
        .build()
        INSTANCE = instance
        instance
    }
}

```

```

private val MIGRATION_1_2 = object : Migration(1, 2) {
    override fun migrate(database: SupportSQLiteDatabase) {
        database.execSQL("ALTER TABLE cats ADD COLUMN gender TEXT NOT NULL
DEFAULT \"\")
    }
}

```

```

private val MIGRATION_2_3 = object : Migration(2, 3) {
    override fun migrate(database: SupportSQLiteDatabase) {
        database.execSQL("ALTER TABLE cats ADD COLUMN specialNeeds INTEGER
NOT NULL DEFAULT 0")
    }
}
}

```

A `@Database`-ben szerepelnek: `entities = [Cat::class]` felsorolja a táblákat, amik az adatbázishoz tartoznak, `version = 3`, ez megadja az adatbázis verziószámát, ha változik a sémája, növeljük ezt a számot, és az `exportSchema = false` pedig kikapcsolja a séma exportálását, így egyszerűsítjük.

A `@Volatile` biztosítja, hogy az `instance` változó minden szálon látható és szinkronizált legyen.

A `synchronized` megadja, hogy csak egy példány legyen.

A `getDatabase()` segít az adatbázis példány lekérésében, ha nincs, létrehozza azt.

A `Room.databaseBuilder()` az adatbázis builder konfigurációja.

A migrációk megadják a változtatásokat, ha változtatunk az entity-n, akkor növeni kell a verziót, és írni egy migrációt hozzá.

Használi példák a kódból:

```

class CatRepository(private val catDao: CatDao) {
    fun getAllAvailableCats(): Flow<List<Cat>> = catDao.getAllAvailableCats()
}

```

```
suspend fun insertCat(cat: Cat): Long = catDao.insertCat(cat)
```

```
suspend fun updateCat(cat: Cat) = catDao.updateCat(cat)
```

```
suspend fun deleteCat(cat: Cat) = catDao.deleteCat(cat)
}
```

Inicializálás:

```
val database = CatDatabase.getDatabase(context)
```

```
val catDao = database.catDao()
```

```
val repository = CatRepository(catDao)
```

ViewModel

Kommunikál a repository-val, StateFlow-ban tárolja a megjelenítendő adatokat, ami azért jó, mert ez egy megfigyelhető állapotot tároló objektum a Kotlin-ban.

```
class CatViewModel(private val repository: CatRepository) : ViewModel() {
    private val _availableCats = MutableStateFlow<List<Cat>>(emptyList())
    val availableCats: StateFlow<List<Cat>> = _availableCats
```

```
    init {
        viewModelScope.launch {
            repository.getAllAvailableCats().collect { cats ->
                _availableCats.value = cats
            }
        }
    }
}
```

Minden változásnál frissíti a UI-t.

Függőségkezelés (Dependency Injection)

Feladata, hogy összekapcsolja a külön rétegeket.

```
val database = CatDatabase.getDatabase(this)
val repository = CatRepository(database.catDao())
val viewModel = CatViewModel(repository)
```

Manuálisan felépíti a függőségi láncot: Database -> DAO -> Repository -> Viewmodel.

Adatok, UI megjelenítés

Feladata az adatok megjelenítése és a user interakcióinak a kezelése.

```
@Composable
fun CatListScreen(viewModel: CatViewModel) {
    val availableCats by viewModel.availableCats.collectAsStateWithLifecycle()
    LazyColumn {
```

```

        items(availableCats) { cat ->
            CatCard(cat = cat, onAdoptClick = { viewModel.adoptCat(cat.id) })
        }
    }
}

```

A Compose automatikusan újrarajzolja a listát, ha a StateFlow megváltozik, ez biztosítja a reaktív UI-t.

Coroutines és Flow

Feladata az aszinkron adatkezelés és az adatok frissítése.

```

viewModelScope.launch {
    repository.getAllAvailableCats().collect { cats ->
        _availableCats.value = cats
    }
}

```

A coroutine háttérszálon fut, a Flow pedig automatikusan jelzi, ha érkezett új adat. Emellett nem blokkolja a fő szálat, így futhat aszinkron módon, anélkül, hogy bármit leállítana vagy késleltetne.

Architektúra

```

UI (Jetpack Compose)
    ↓
ViewModel
    ↓
Repository
    ↓
DAO (Room)
    ↓
SQLite Database

```

Az alkalmazás MVVM architektúrát használ, ahol az adatok Room-ból indulnak és a Compose felületen jelennek meg, ahol automatikusan frissülnek.

Design

Színpaletta

```

package com.vm3dir_mobilprog_catshelter.ui.theme
import androidx.compose.ui.graphics.Color
val PrimaryPink = Color(0xFFFF8BBD9)
val LightPink = Color(0xFFFFCE4EC)
val AccentPink = Color(0xFFFF48FB1)
val DarkPink = Color(0xFFE91E63)

```

```

val SoftWhite = Color(0xFFFFF8FC)
val TextPink = Color(0xFFAD1457)
val Pink100 = Color(0xFFF8BBD9)
val Pink200 = Color(0xFFF48FB1)
val Pink300 = Color(0xFFE91E63)
val Pink400 = Color(0xFFC2185B)
val Pink500 = Color(0xFFAD1457)

```

A színeket globálisan hoztam létre, hex formátumban tároltam el őket, így könnyebb volt felhasználni a projektben.

Téma konfigurációja

```

private val LightColorScheme = lightColorScheme(
    primary = PrimaryPink,
    secondary = AccentPink,
    tertiary = DarkPink,
    background = LightPink,
    surface = SoftWhite,
    onPrimary = TextPink,
    onSecondary = TextPink,
    onTertiary = SoftWhite,
    onBackground = TextPink,
    onSurface = TextPink
)

```

```

@Composable
fun VM3DIR_MobilProg_CatShelterTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else
dynamicLightColorScheme(context)
        }
    }
}

```

```

        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }
}

```

```

MaterialTheme(
    colorScheme = colorScheme,
    typography = Typography,
    content = content
)

```

```
}
```

A lightColorScheme és a darkColorScheme színsémák között váltogat.

Betűtípusok

```
val Typography = Typography(  
    bodyLarge = TextStyle(  
        fontFamily = FontFamily.Default,  
        fontWeight = FontWeight.Normal,  
        fontSize = 16.sp,  
        lineHeight = 24.sp,  
        letterSpacing = 0.5.sp  
    )  
)
```

UI komponensek

Card

```
Card(  
    modifier = modifier  
        .fillMaxWidth()  
        .padding(horizontal = 16.dp, vertical = 8.dp),  
    shape = RoundedCornerShape(16.dp),  
    colors = CardDefaults.cardColors(  
        containerColor = SoftWhite  
    ),  
    elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)  
) {  
    // Card tartalom  
}
```

Színhasználat a komponensekben

```
Button(  
    colors = ButtonDefaults.buttonColors(  
        containerColor = AccentPink, // Gomb háttér  
        contentColor = SoftWhite    // Gomb szöveg/ikon színe  
    )  
)
```

```
OutlinedButton(  
    colors = ButtonDefaults.outlinedButtonColors(  
        contentColor = AccentPink  
    ),  
    border = BorderStroke(1.dp, AccentPink)  
)
```

```
OutlinedTextField(  
    colors = OutlinedTextFieldDefaults.colors(  
        // ...  
    )  
)
```

```

        focusedBorderColor = AccentPink,    // Aktív állapot keret
        unfocusedBorderColor = Pink200,    // Inaktív állapot keret
        focusedLabelColor = AccentPink,    // Aktív címke színe
        unfocusedLabelColor = TextPink,    // Inaktív címke színe
        errorBorderColor = MaterialTheme.colorScheme.error,
        errorLabelColor = MaterialTheme.colorScheme.error
    )
)

```

```

Surface(
    color = AccentPink,
    shape = RoundedCornerShape(12.dp)
) {
    // Tartalom
}

```

```

Icon(
    imageVector = Icons.Default.Pets,
    tint = TextPink, // Ikon színe
    modifier = Modifier.size(24.dp)
)

```

Reszponzív design: Modifier

```

Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.SpaceBetween
) {
    Column(modifier = Modifier.weight(1f)) {
        // Tartalom
    }
    Column(modifier = Modifier.weight(1f)) {
        // Tartalom
    }
}

```

```

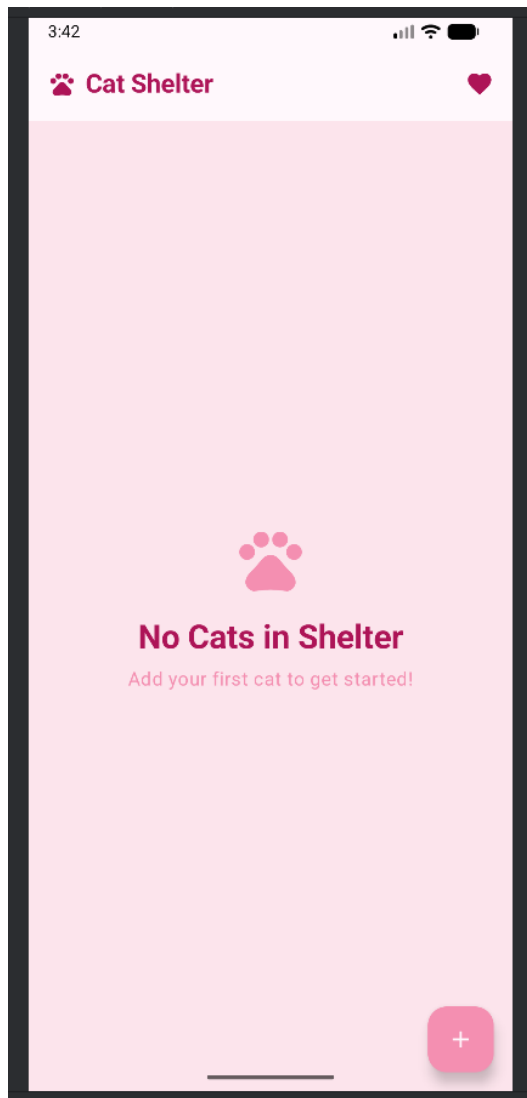
Box(
    modifier = Modifier
        .background(
            color = if (cat.specialNeeds) AccentPink else LightPink,
            shape = RoundedCornerShape(6.dp)
        )
)

```

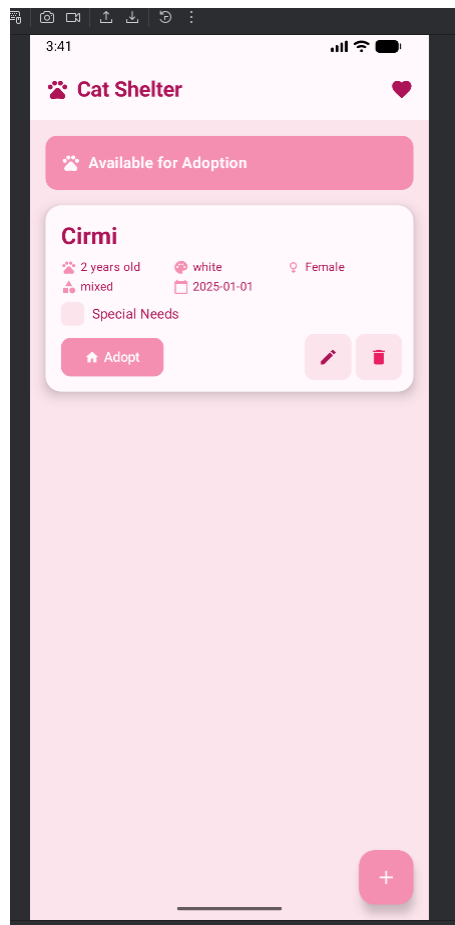
Arányos elosztást tesz lehetővé, minden oszlop ugyanannyi helyet kap, térközt is beállítja az elemek között, feltételes színezést tesz lehetővé az if kifejezéssel, és háttérszínt is beállít.

Képernyőképek

Főoldal



Amikor üres, nincs felvéve adat, visszajelez számnkra egy szöveg, hogy nincsenek egyelőre macskák a menhelyen, a plusz gombra kattintva lehet hozzáadni új cicát.



Ha hozzáadtunk, egy kártyában megjelenik a neve, és az adatok, amiket a form-ban kitöltöttünk.

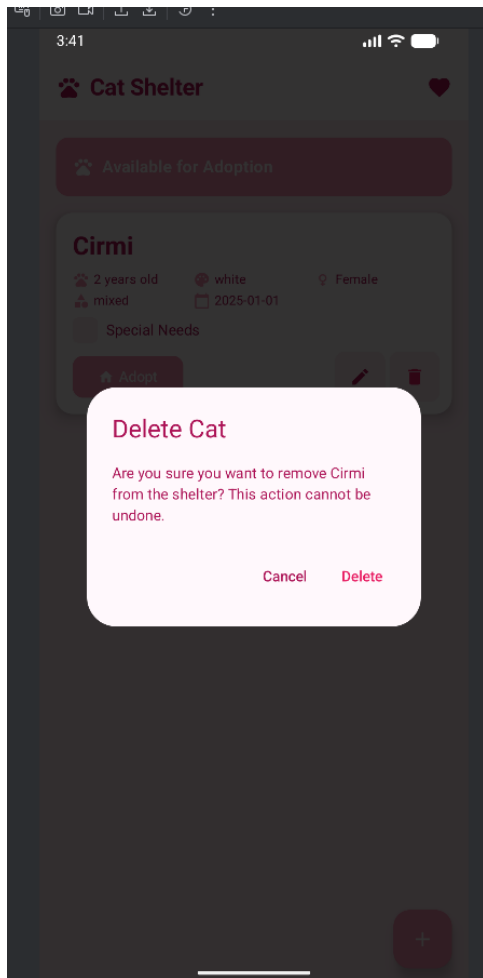
Macska hozzáadás/szerkesztés form

Így néz ki a form, amit ki kell tölteni, ahhoz hogy hozzáadjunk egy új cicát. Így néz ki a szerkesztés is, az adott cica adataival feltöltve.

Ha hozzá akarunk adni egy cicát, és egy olyan input mezőt nem töltöttünk ki, ami kötelező, akkor egy hibaüzenet fogad minket.

The image displays two side-by-side screenshots of a mobile application interface for adding a new cat. Both screenshots show the 'Add New Cat' form, which is a light blue card with a rounded rectangle shape. The form is set against a dark blue background with a subtle grid pattern. The top of the app shows the status bar with the time (4:37 on the left, 3:41 on the right) and signal/battery icons. The app's header is 'Cat Shelter' with a paw print icon on the left and a heart icon on the right. The form itself has a title 'Add New Cat' in bold black text. Below the title, there are several input fields: 'Cat Name*' (with a paw print icon), 'Age' (with a calendar icon), 'Breed' (with a cat icon), 'Color*' (with a cat icon), 'Gender*' (a dropdown menu showing 'Female'), 'Arrival Date (YYYY-MM-DD)*' (with a calendar icon), 'Description' (with a document icon), 'Medical Notes' (with a medical bag icon), and 'Special Needs' (a checkbox). At the bottom of the form are two buttons: 'Cancel' and 'Add Cat'. In the right screenshot, a red error message 'Name is required' is displayed above the 'Cat Name*' field. A white circular button with three horizontal lines is also visible on the left side of the right screenshot.

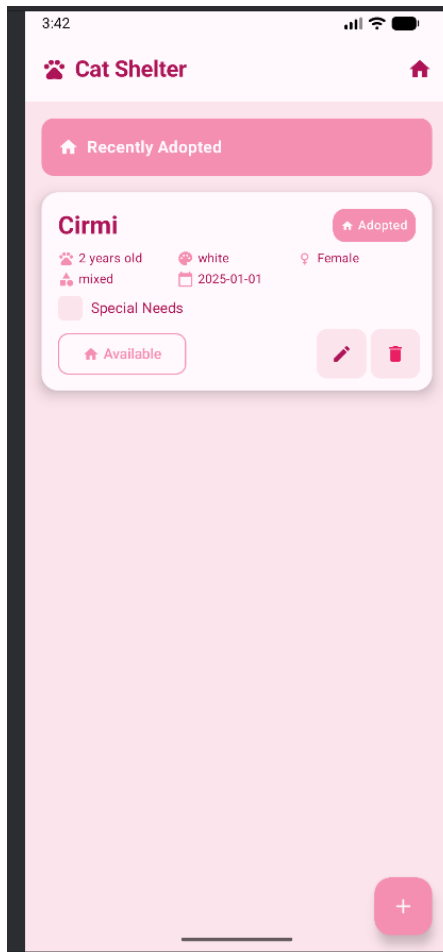
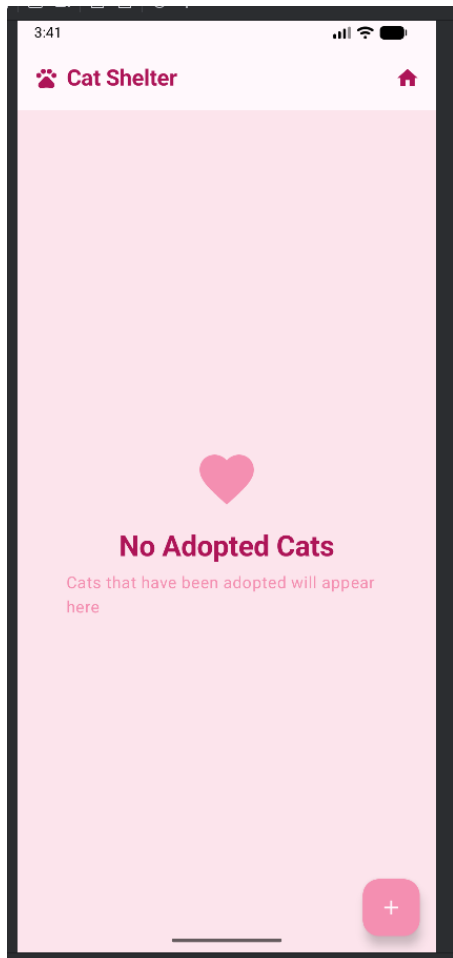
Figyelmeztető üzenet



Ha hozzáadtunk egy cicát, és azt törölni akarjuk, kapunk egy Are-You-Sure felugró üzenetet, hogy biztosan végre akarjuk-e hajtani a törlést.

Adoptált macskák oldal

A jobb felső sarokban a szív ikonra kattintva eljuthatunk az adoptált cicák részlegre, a házikóra kattintva visszajuthatunk a főoldalra, és a még befogadásra váró cicák listájához.



Ha egy macska kártyájánál rámentünk a főoldalon, hogy “Adopt” ebbe a szekcióba átkerülnek a cicák, egy badge-vel megjelölve, hogy már örökbe lettek fogadva, itt is törölhetünk, módosíthatunk a cicán, ha akarunk, illetve az “Available” gombra kattintva visszakerülhet a cica a menhelyre.